

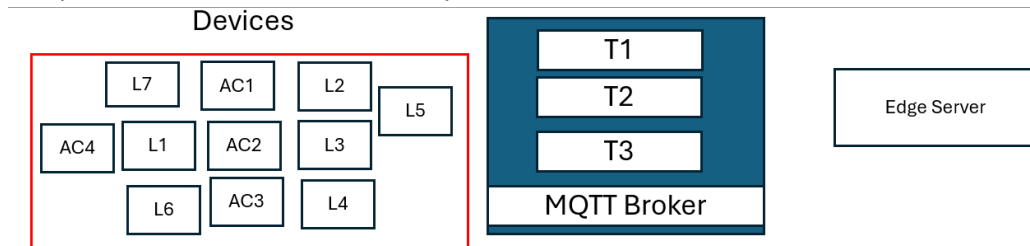
Hands on: Create Smart home

In this hands-on we'll create a simple Smart Home setup. The main focus here will be on designing communication channels and payloads over MQTT, and storage data structures on the edge server.

1. Imagine a home with multiple defined 'rooms' and every room have devices like Smart AC & Smart Light

BR1	AC3 L3	K
L1 AC1	LivingRoom	L6
BR2		Garage
L2 AC2	AC4 L4 L5	L7

2. Common edge server which can bidirectionally interact with each of the devices using an MQTT.
3. Each device can push to, and listen to, various topics on the MQTT server.



4. The edge server can also receive commands from consumers which we'll simulate here by calling methods on the server from a main.py driver file.

Additional Note

- The underlying communication layer can be WiFi, bluetooth, etc. but you do not have to worry about that in these hands on, as you'll be running (simulating) all of them on your local machine.
- For simplicity we can assume that components do not require any authentication or access control.

Components

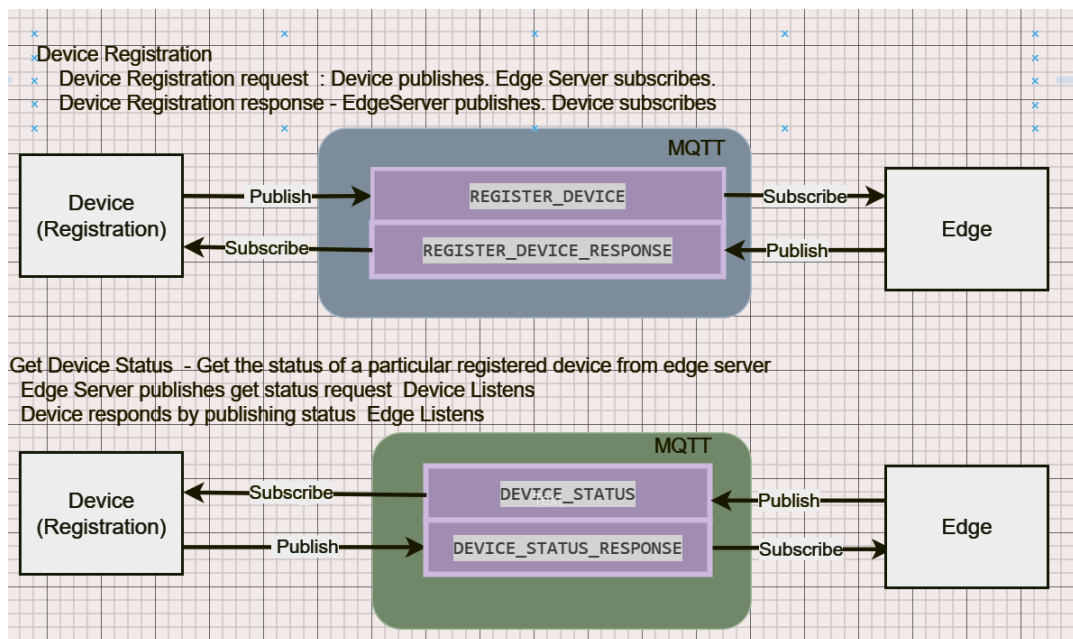
1.	MQTT server	▪ Set up an MQTT server with no authentication, ▪ The ip and port could be specified in a config and read by all components OR passed to all devices and edge server instances during initialization.
2.	Rooms & Device Type	▪ Assume a predefined list of rooms in the home - Kitchen, BR1, BR2, Living & Garage. ▪ Assume a predefined list of device types - LightDevice, ACDevice. ▪ Each device is mapped to a room. All device/room mapping is maintained in Edge Server.
4.	LightDevice	Light can have multiple-intensity light (off/low/medium/high). It should be able to ▪ register with the server, ▪ switch on/off , change state to one of 4 values based on command from the server, and ▪ respond to current status requests.
5.	ACDevice	Simple AC with on/off switch, and temperature control (18 °C - 32 °C). It should be able to ▪ register with the server, ▪ switch on/off, set temperature directly based on command from the server keeping check on the bounds, it can send an error response on out of bounds requests. ▪ respond to current status requests.
6.	Edge server	▪ It should be able to store the registered devices, For simplicity we'll not use any DB and all info will be stored only in memory as array/list. ▪ accept new registrations, ▪ and send commands to the devices based on calls made to its external interface.
7.	Driver	▪ This is consumer simulator (main.py) which creates appropriate instances and initiates calls on the server to highlight the capabilities.

Program Organization

- You are provided with the starter code for device classes, edge server class and main.py to get started.
- Please feel free to modify the variables, functions and calls as you prefer.
 1. main.py: This is the driver code(simulator for consumer) where you will be creating different devices and performing various operations related to the functionality of the devices. The main.py should cover all the cases mentioned in the problem statement.
 2. ACDevice.py: This file will contain all the operations and methods relevant to the AC device_type. This will have the implementation of different methods such as registration, setting the temperature and fetching the temperature.
 3. LightDevice.py: This file will contain all the operations and methods relevant to the Light_Device, device_type. This will have the implementation of different methods such as registration, setting the intensity of the light_device and fetching the current intensity status of the device.
 4. EdgeServer.py: This is the main file which will get the command from the user to perform various operations for each of the devices. The different methods in EdgeServer class will help in making the communication with the devices to perform the task given by the user.
- **Key Housekeeping points**
 1. This hands on is just for learning purpose and may not follow some standard practices.
 2. The focus is on the main flow, with relatively minimal error handling.

Problem Statement

- Demonstrate at least 3 lights and 2 ACs, spread across at least 3 rooms. All communication between device and Edge will happen via MQTT
- Key concept here would be design **topic topology** for various types of communication. Following needs to be kept in mind -
 1. You don't use the same topic to send information back-and-forth. Being a publisher and subscriber on the same topic is not ideal due to unnecessary communication and lack of clarity.
 2. Imagine two extremes,
 - one where each device has its own unique topic to publish and one to subscribe. In this case, logic to communicate is simple but the server gets loaded as for bulk calls, it has to write to, and read from, many topics.
 - Another extreme is having only one topic to transmit and one to receive from the server. All devices can listen to that topic and only act on information that is for them, based on device id in the payload. This is very easy for the server but now each device is potentially getting a lot of unnecessary information to parse. It'll also hamper access control and make privacy separation among devices harder
 3. Therefore you need to see how to design topics to balance server and device load. You can choose to implement one of the extremes or a healthy mix where each device might interact on its own unique set of channels but also listens to common channels for its type and location, as an example.
 4. Below is example how the communication between 2 components should happen -



Task : Consumer (main.py) can perform following basic actions

1. Register a new device (all the devices should be registered on edge server)
2. Get list of registered devices
3. Get status of any device , or group of device by device type/room, all
4. Set status of any device , or group of device by device type/room, all
5. Set value of device (intensity of light or temperature of ac)

To accomplish above following needs to be done

- Implement methods in the AC_Device, Light_Device and Edge_Server and registration
 1. Implement registration method with the server
 - a. Implement logic for new Device registration with device id, device type, and room type, at least. MQTT Registration topic can be hard-coded or passed on to the constructor.
 - b. On receiving the registration call, the server should store the device details in its registered device list, and respond back with a confirmation on another appropriate topic.
 2. Implement status and switch on/off commands
 - a. Implement a status command on the server and devices
 - i. Implement get_status method exposed into the main.py driver. This can be called based on any of the following - device id, device type (all devices of that type), room type (all devices in that room), 'all' (all devices in the home).
 - ii. The server should appropriately call all relevant devices and get status back from them. It should then collect responses and return to main.py.
 - iii. Each device should implement a status method returning the status of their dynamic values (light state, switch, temperature, etc.)
 - b. Implement commands to switch off/on devices, on the server and devices
 - i. The set status method(s) that are exposed to the main.py driver to initiate switch on/off commands. This can be based on any of the following - device id, device type (all devices of that type), room type (all devices in that room), 'all' (all devices in the home)
 - ii. The server should appropriately call all relevant devices methods and get command status back from them. It should then collect responses and return to main.py. Relevant device class should have a method for getting the status.
 - iii. Implement the set methods in each of the device classes to switch off/on method(s) to change their status, returning success or failure accordingly. Of course, 'off' here is equivalent to reducing electricity consumption but they'll continue to listen to new commands.
 3. Implement control commands for both devices
 - a. Implement light intensity control command - change to a specific value (off/low/medium/high)
 - i. Implement the set method exposed to the main.py driver to initiate a light control command. This can be based on any of the following - device id, room type (all light devices in that room), 'all' (all light devices in the home)
 - ii. The server should appropriately call all relevant devices and get command status back from them. It should then collect responses and return to main.py. Relevant device class should have a method for getting the status.
 - iii. Implement the given control command method in Light_Device class, implementing the

change and returning success or failure accordingly.

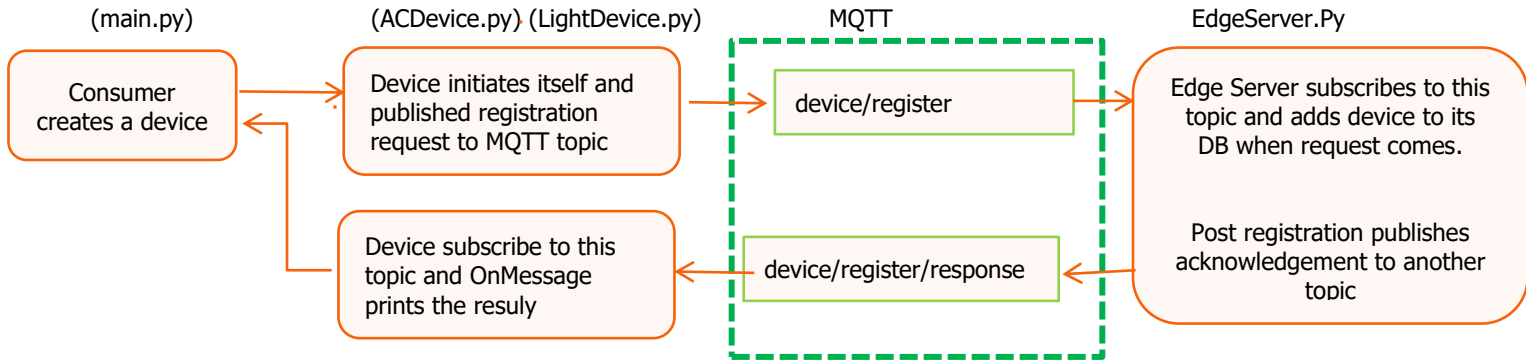
- b. Implement AC control command - change the temperature between (18 °C - 32 °C)
 - i. Implement the set method exposed to the main.py driver to initiate an AC control command. This can be based on any of the following - device id, room type (all AC devices in that room), all (all AC devices in the home)
 - ii. The server should appropriately call all relevant devices and get command status back from them. It should then collect responses and return to main.py. Relevant device class should have a method for getting the status.
 - iii. Implement the given control command method in AC_Device class, implementing the change and returning success or failure accordingly.

Note:

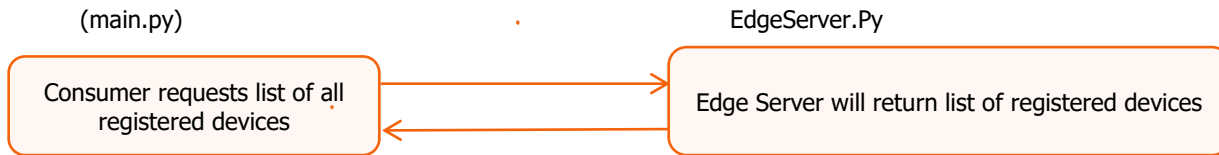
- The base code files related to ACDevice, LightDevice and EdgeServer are provided.
- You can go ahead and start implementing them after understanding the problem statement.
- Also, If you want to start from scratch you can do that.
- You are supposed to write code that should mimic the behavior of a smart home.
 - New devices can be registered on the server
 - Server can communicate with devices for various command executions basis input from consumer.
 - Consumer can send commands on different parameter based on devices, device type, room and home.
- It is important that you are using MQTT to perform the communication between server and devices.
- Once the commands are executed by devices, the server should return a response back to the main, confirming about the same. This task is important and it is part of the problem statement 2.a.ii, 2.b.ii, 3.a.ii and 3.b.ii.
- Expected work to be done by you:
 - ACDevice.py: This file will contain the implementation for ACDevice type. It will contain methods to listen on various topics and then it will have implementation to perform get and set operations based on the received command.
 - LightDevice.py: This file will contain the implementation for LightDevice type. It will contain methods to listen on various topics and then it will have implementation to perform get and set operations based on the received command.
 - EdgeServer.py: This is a class that will invoke and have registration processing, get and set methods that will publish the requests for processing topics based on received commands.
 - main.py: This will be the driver code that should be invoked to perform all the commands related to registration and command execution.
- Expected sample output file is also provided in the zip folder. The file has multiple test cases that will help you in designing and understanding the different output for different commands. It is not required to match your print statements. Your goal should be to ensure that your code is covering at least these test cases.

Use Case and Flows

1. Device Registration



2. Get List of all Devices



3. Get status & Set Status (of a Device / Device Type / Room / all)

